SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

XXX_PocketCFO

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

PocketCFO is an AI-powered decision intelligence platform designed to tackle the critical financial management bottlenecks faced by businesses and gig workers in Malaysia. At its core, the system leverages Z.AI GLM to process fragmented, unstructured financial data—such as conversational inputs and raw receipts—and transform it into actionable business strategies. By shifting the focus from simple data entry to context-aware reasoning, PocketCFO empowers users to optimize their tax reporting, protect their cash flow, and make informed economic decisions without needing an extensive background in accounting.

1.1. Purpose:

The System Analysis Document highlights the technical scope and design-related decisions behind the PocketCFO development. The key elements of the system that have been covered are as follows:

1. **Architecture:**
   a. The application is built on a client-server model utilizing the Next.js and React.js framework.
2. **Data Flows:**
   Unstructured user inputs (such as raw expense texts or receipt details) are captured via the client-side chat interface, routed securely through the Next.js backend, and processed by the Z.AI GLM. The resulting insights are then structured and relayed back to the user's dashboard in real time. Strategic recommendations are suggested and the graph shows a decision making process based on the inputs.
3. **Model Process:**
   The system's end-to-end workflow encompasses capturing financial transactions, contextualizing the data against current tax regulations via Z.AI GLM , purchase timing optimization, tax categorization and outputting proactive, quantifiable financial recommendations.
4. **Role of Reference:**
   This document acts as the definitive technical baseline for software engineers, prompt developers, judges, and quality assurance testers to ensure alignment on system capabilities and API contracts.

1.2. **Background**:

Small and Medium Enterprises (SMEs) in Malaysia often operate with fragmented financial data and limited analytical capabilities. The founders of PocketCFO identified a market gap where business decisions—such as expense categorization and cash flow forecasting—are largely manual or based

XXX_PocketCFO

on intuition rather than data-driven intelligence. This reliance on manual processes often leads to missed opportunities for cost control and tax optimization.

1. Previous Version
   Traditionally, business owners used standalone, manual tools such as physical ledger books or basic spreadsheet software (e.g., Excel). These previous versions of financial management were limited to simple automation or data entry. While they could record a transaction, they lacked the intelligence to analyze information or support strategic decision-making, leaving the user with the burden of interpreting complex financial patterns alone.

2. Changes in Major Architectural Components
   The development of PocketCFO introduces a fundamental shift from deterministic rule-based software to an AI-driven decision intelligence architecture, which is the integration of the Z.AI GLM.

3. New capabilities that are only introduced in this version

The following capabilities are introduced in this version to improve economic outcomes:

a. Context-Aware Recommendation: The system does not just record data; it recommends actions, such as purchase timing or cost control strategies.
b. Interpretation of Unstructured Data: The ability to process raw financial inputs and generate structured insights using Z.AI's GLM.
c. Clear Explanation of Decisions: Every strategic recommendation is accompanied by a user-understandable explanation of the underlying logic.
d. Quantifiable Impact Tracking: Features designed to measure and display improvements in revenue, time saved, or cost reduction

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
SME Owner	Primary end-users who interact with the chat interface to upload receipts and query financial strategies.	An intuitive conversational interface, accurate expense categorization, and actionable advice that yields quantifiable cost reductions or tax savings.

XXX_PocketCFO

Development Team	Responsible for building the Next.js platform and integrating the Z.AI GLM	Clear API usage contracts, modular codebase, and a well-documented architecture.
QA TEAM	Validating the system behavior for all user roles, specifically the accuracy of the AI's financial reasoning.	Defined scope, clear request/response formats, reliable UI performance under various user inputs. and coverage of edge cases.

XXX_PocketCFO

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web-based Financial Assistant (Next.js Application)
Architecture	Client-Side App Router with Integrated GLM Service Layer

PocketCFO is a modern web application designed as a personal financial officer. It follows a Client-Server architecture leveraging Next.js. The frontend (React components) handles user interaction and data visualization, while the backend (Next.js API routes) manages communication with the GLM. It is designed to be lightweight, with the core logic residing in an "engine" that parses financial data and maintains state within the user's browser session.

2.1.2 GLM as Service Layer

- Z.AI GLM is the core reasoning engine for Decision Intelligence, converting unstructured financial data into actionable business strategies.
- It enables economic empowerment for users like Malaysian SMEs and gig workers to optimize tax reporting and cash flow.
- The system provides quantifiable financial recommendations for cost control and revenue increase.

2.1.3 Dependency Diagram

a. **Prompt Construction:** The system takes raw user input (text) and combines it with a **System Instructions** template that defines the persona (CFO) and current financial context (state).

b. **Context Window:** The context includes the last 5-10 chat messages (`messages` state) and summarized "Graph State" data (e.g., "IKEA Receipt", "RM7,850.00") to ensure the AI "sees" the charts the user sees.

c. **Parsing & Flow:** The backend receives a JSON response from the GLM. It is parsed by a **Response Handler** that separates the "Advice" (text) from "Actions" (e.g., updating the expense graph).

d. **Limitations & Chunking:** Before sending to the API, a **Token Utility** checks the input length. If a user uploads a massive CSV/Statement, the system chunks the data into 2000-token blocks using a sliding window approach.

© UMHackathon 2026

Major API Calls

- **Browser → Next.js API:** `POST /api/chat` (Sends user message + current cashflow state).
- **Next.js API → GLM API:** (Sends constructed prompt with API Key authentication).
- **Next.js API → Database:** `GET /user/profile` (To fetch historical spending patterns for better context).

2.2.2. Sequence Diagram

a. User: Input "I just spent RM50 on coffee" in the chat.

b. Frontend: Updates the message state to show the user's text.

c. Backend Service: Receives the text, identifies it as an expense, and constructs a prompt.

d. GLM: Processes the input and returns a structured JSON: {"category": "Food", "amount": 50, "response": "Noted! Your 'Food' budget is now 80% used."}.

e. Frontend Logic: Updates the Graph State to reflect the new total and displays the "bot" response in the chat window.

2.2. Technological Stack

2.2.1. Frontend: Next.js (App Router) + TypeScript – Provides a high-performance, minimalist UI with fast routing.

2.2.2. Backend: Next.js Server Actions / API Routes – Handles logic without needing a separate Express server.

2.2.3. Database: Firebase

2.2.4. Deployment: Vercel

2.3. Key Data Flows

2.3.1.1. Data Flow Diagram (DFD)

a. External Entities: User (Input/View), Bank API (Optional data source).

b. Processes: Transaction Categorization, Trend Analysis, LLM Prompt Generation.

c. Data Stores: User Session (Local State), Transaction Database (Persistent Storage).

2.3.2. Normalized Database Schema

a. Users Table: user_id (PK), email, name, created_at.

b. Transactions Table: tx_id (PK), user_id (FK), amount, category_id (FK), timestamp.

c. Categories Table: category_id (PK), name (e.g., Food, Rent, Salary), budget_limit.

XXX_PocketCFO

3. Functional Requirements & Scope

This section outlines the primary functionalities designed to showcase the "Decision Intelligence" capabilities of PocketCFO during the UMHackathon 2026.

3.1. Minimum Viable Product:

The following 3 core features represent the PocketCFO MVP that will be demonstrated by the end of the competition:

#	Features	Description
1	Unstructured Expense Entry	Users can input business expenses (e.g., "Bought a new oven for RM3,000") via a chat interface.
2	Strategic Recommendation Engine	The system analyzes inputs and current cash flow to provide actionable advice, such as recommending a purchase delay to optimize tax brackets.
3	AI-Powered Dashboard Visualization	A visual dashboard (mockup) that displays real-time cash flow health and quantifiable impacts generated by the AI.

3.2. Non-Functional Requirements (NFRs)

These requirements highlight the system qualities essential for reliability and operational excellence:

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Decision Intelligence	The system must provide actionable insights beyond simple data entry automation.	Z.AI GLM serves as the core reasoning engine for all financial advisory logic.
Security	Sensitive API keys and financial logic must be protected from public exposure.	Next.js Backend API Routes are used to handle all communication with Z.AI GLM securely.
Token Latency	API calls to the GLM must return a response within a professional timeframe (e.g., < 3s-5s).	Implementation of asynchronous API calls and user-facing loading states to improve perceived speed.

XXX_PocketCFO

Cost Efficiency	Average token costs per session must remain within defined budget thresholds.	Use of fixed-length system prompts and truncation of user inputs exceeding 1,500 tokens.

3.3. Out of Scope / Future Enhancements

These high-level features are part of the "Grand Vision" for PocketCFO but are not included in the initial UMHackathon 2026 submission:

a. Real-time LHDN Integration: Direct automated filing to actual LHDN servers (simulated in MVP).
b. Multi-User Collaboration: Simultaneous real-time editing and input by multiple business partners.
c. Automatic Bank Syncing: Direct API connections to Malaysian banking institutions for automated transaction imports.

XXX_PocketCFO

XXX_PocketCFO

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. **Grayscale Rollout & A/B Testing**: New AI advisory features, such as "Tax Optimization Logic," are released to 5% of users initially. Performance is monitored before a gradual increase to 100%. A/B testing is then used to compare different Z.AI System Prompts to see which provides clearer financial advice to SME owners.

4.1.2. **Emergency Rollback (ER) & Golden Release**:If anomalies are detected in the AI response accuracy or system latency, an ER is triggered. The automated Vercel pipeline immediately deploys the Golden Version (the last stable build) of PocketCFO to ensure business continuity.

4.1.3. Priority Matrix:

4.1.3.1. The following matrix defines trigger conditions and required actions for system stability:

Priority Level	Trigger Condition	Action Taken

© UMHackathon 2026

P1 - Critical	Z.AI GLM total downtime or failure rate > 5%.	Immediate Emergency Rollback and notification to users of a "Maintenance State".
P2 - High	AI response latency exceeds 10 seconds for multiple sessions.	Trigger prompt optimization logic and increase truncation limits to reduce token load.
P3 - Medium	Minor UI rendering issues on the "Cashflow Health" dashboard.	Log defects in the project board and fix it in the next scheduled sprint.

4.2. Monitoring:

4.2.1. PocketCFO utilizes automated monitoring to track the "health" of the system. If AI output pass rates drop below the 80% threshold, an alert is triggered to the development team to refine the System Prompt logic.

4.3. Assumptions:

a. Internet Stability: Users are assumed to have a stable internet connection while interacting with the chat-based interface.

b. Data Integrity: It is assumed that the SME owner provides truthful and legible text-based financial inputs for the AI to analyze.

c. Device Access: Business owners have access to a web-enabled device to view the high-class dashboard.

4.4. **External Dependencies:** Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks

XXX_PocketCFO

Gemini API	The mandatory core reasoning engine for prompt inference and generating financial decisions.	High: API rate limitations or downtime will cause core product failure. Mitigation: Implement graceful degradation and error messages.

XXX_PocketCFO

5. Project Management & Team Contributions

The development of Pocket CFO followed a structured 10-day sprint, aligning with the duration of the Preliminary round. This timeline ensures that all deliverables, including the prototype and documentation, were developed exclusively during the official competition period.

5.1. Project Timeline:

Days	Phase	Activities
Day 1-2	**Project Planning**	Problem statement analysis for Domain 2, requirement gathering, and defining the **Pocket CFO** vision.

Day 3-4	**System Analysis**	Drafting the SAD, designing the 3NF database schema, and mapping Z.AI GLM as the service layer.
Day 5-7	**MVP Development**	Coding the Next.js frontend, setting up backend API routes, and performing prompt engineering for Z.AI.
Day 8-9	**QA & Testing**	Executing unit and integration tests, performing AI boundary testing, and risk mitigation.
Day 10	**Final Submission**	Conversion of all documentation, recording the 10-minute pitching video, and GitHub repository finalization.

5.2. Team Members Role:

a. **Team Leader (Wong Ming Jian)**: Primary point of contact for the organizer and responsible for final deliverable submission.
b. **Frontend Lead (Lim Zhi Ying)**: Responsible for the "high-class" Next.js UI/UX design and dashboard visualization.
c. **Backend & AI Lead (Vincent Chew Wen Jie)**: Responsible for Next.js API routes, Z.AI GLM integration, and system prompt logic.
d. **QA & Documentation Specialist (Chong Chai Yee)**: Responsible for drafting the PRD, SAD, and QATD, and managing version control.

5.3. Recommendations:

a. Performance: Implement Redis Caching for frequently requested LHDN tax categories to reduce Z.AI GLM call frequency and latency.
b. Reliability: Transition to a Microservices architecture as the user base grows, separating the advisory engine from the data ingestion service.
c. Scalability: Utilize AWS Auto Scaling Groups (ASG) to manage traffic spikes during peak business hours (e.g., month-end tax filing periods).

xxx_PocketCFO